

Disk Image Forensics

Imaging · FTK Imager · \$MFT · deleted files · timestamps

Stefanus Eko Prasetyo

Lab-6 · hands-on session

```
PS C:\Tools>
PS> MFTECmd.exe -f $MFT --csv .
    40 records parsed
note.txt ..... rec 40
# the disk remembers
```

Today's roadmap

1

Why images, not originals

bit-by-bit copies, hash verification, chain of custody

2

Acquiring an image

FTK Imager step by step - raw dd and AD1 formats

3

Analyzing an image

the four FTK panes: tree, properties, file list, preview

4

NTFS metadata files

\$MFT, \$MFTMirr, \$LogFile, \$Boot - the filesystem's own diary

5

Recovering evidence

deleted files, alternate data streams, and honest timestamps

6

Challenges & working paper

upstream USB image + two new challenges + documentation rules

The disk remembers everything

A disk image is a **bit-by-bit copy** of an entire device - not just the visible files, but the empty space, the deleted content, and the filesystem's own bookkeeping. Working on a verified copy is how evidence stays admissible.

Visible files

what any user can browse - the smallest layer of the story

Deleted content

deleted does not mean gone - the bytes usually survive

Filesystem metadata

\$MFT records, timestamps, ADS - the diary the disk keeps

Unallocated space

slack, fragments, and carved artifacts with no owner

Three words, one discipline

Term	Meaning
disk image	a bit-by-bit copy of a device or partition, including deleted data and metadata
disk imaging	creating that copy under hash verification, so the original is never touched
image forensics	analyzing the copy: files, registry, browser data, logs, deleted artifacts

The rule that makes it forensic: hash at acquisition, work only on copies, hash again after analysis. Identical hashes prove the original was never modified - that is chain of custody in one sentence.

Creating an image with FTK Imager

- 1 File > Create Disk Image, select Physical Drive
- 2 Pick the device (a USB drive in our example)
- 3 Add > image destination - choose Raw (dd)
- 4 Enter evidence details: case, examiner, notes
- 5 Start - then record the reported hashes

Format field guide

- ▶ Raw (dd): every bit, no metadata, universally readable - our Challenge B
- ▶ AD1: FTK's logical format - what the upstream Image.ad1 uses
- ▶ E01: EnCase format with compression + built-in hashes
- ▶ Whatever the format: verify the hash before you analyze

FTK Imager's four panes

Pane	Where	Shows
Evidence Tree	top left	the image's hierarchy - expand to walk the filesystem
File List	center top	files and folders of the selected node, with sizes and times
Properties	bottom left	the selected file: timestamps, and its MD5 / SHA1 hashes
Preview	bottom center	the file's content - text, image, or hex view

Add Evidence Item > Image File to open a captured image. Right-click any file > Export Files to extract it - and hash your exports: they are evidence now.

The filesystem's own files

File	Role	Forensic value
\$MFT	master file table: one record per file and folder	names, times, parents, deleted records
\$MFTMirr	backup copy of the first \$MFT records	recovery when \$MFT is damaged
\$LogFile	journal of metadata transactions	what changed, and when
\$Boot	the volume boot sector	geometry, and the volume serial number
\$Bitmap / \$Volume / \$Extend	cluster map / label / extensions	allocated vs free, system structure

\$MFT — one record, one file

PS C:\Tools>

```
record 40, 1024 bytes:
FILE0 ... flags=0x0001 (in use)
  attr 0x10 $STANDARD_INFORMATION -> SI times
  attr 0x30 $FILE_NAME             -> name + PARENT
      name: note.txt   parent: rec 5 (root)
  attr 0x80 $DATA             -> content

# the record NUMBER is the file's identity;
# the PARENT ref rebuilds the folder tree
```

Why examiners love it

- ▶ every file gets a numbered record
- ▶ deleted records often survive in place
- ▶ parent refs rebuild directory structure
- ▶ resident data lives inside the record

Reading \$MFT: MFTECmd and analyzeMFT



PS C:\Tools>

```
# export $MFT from FTK, then:
```

```
PS> MFTECmd.exe -f "C:\evidence\${MFT}" --csv C:\evidence out.csv  
-> record number, name, parent, SI and FN timestamps, size
```

```
# python alternative:
```

```
$ python analyzeMFT.py -f $MFT -o records.csv
```

```
# filter the CSV: find a name, follow its parent ref,
```

```
# compare SI vs FN timestamps (next slides!)
```

Deleted is not gone

Filesystem	What deletion does	What you can recover
FAT (USB sticks)	directory entry first byte -> 0xE5, FAT chain cleared	entry text, then the content if clusters are untouched
NTFS	MFT record flagged free; data location kept until reuse	the whole record - name, times, content pointers

FTK Imager shows deleted entries in the tree; hex view confirms the 0xE5 marker; Export or carve the content from the data area.

Timestamps - and timestomping

Attribute	Written by	Trust level
\$STANDARD_INFORMATION (SI)	user-space tools - what Explorer shows	EASILY FAKED
\$FILE_NAME (FN)	the kernel, on the parent directory's record	hard to fake

Timestomping = backdating a file by rewriting SI times. The tell: SI and FN timestamps disagree wildly (a file 'created in 2019' whose kernel-recorded name says 2026). Always compare both - a mismatch is a finding, not a glitch.

Alternate data streams — the hidden side

PS C:\Tools>

```
# meme.jpeg:Zone.Identifier (exported from the image):  
[ZoneTransfer]  
ZoneId=3  
ReferrerUrl=https://twitter.com/  
HostUrl=https://pbs.twimg.com/media/FadAHV...  
  
# NTFS files can carry named streams;  
# browsers write the download origin into this one
```

Why it matters

- ▶ invisible in Explorer and dir listings
- ▶ carries the ORIGINAL download URL
- ▶ malware hides payloads in streams
- ▶ FTK lists them under the file in the tree

FAT vs NTFS — two worlds on your desk

	FAT16/32	NTFS
Where	USB sticks, SD cards	system drives (Windows)
Bookkeeping	FAT tables + fixed root folder	\$MFT record per file
Deletion mark	0xE5 in the directory entry	record flagged not-in-use
Timestamps	created/written/accessed, 2s resolution	SI + FN, 100ns resolution
Extras	no streams, no journal	ADS, \$LogFile journal, security

Same discipline on both: image first, walk the bookkeeping structures, recover what deletion forgot to erase.

The disk-image toolbox

Tool	Purpose
FTK Imager	acquire images, browse, preview hex, export files and ADS
MFTECmd	parse \$MFT / \$Boot / \$LogFile into CSV timelines
analyzeMFT	Python alternative for \$MFT parsing
Autopsy	heavyweight analysis: timelines, recovery, reports
HxD / hex editor	ground truth - read bytes when tools disagree

Your challenges

Challenge A

the USB image

`files/Image.ad1`

- ▶ 5 questions, no flags
- ▶ hashes, record numbers, parents
- ▶ a tweet's origin, a boot serial
- ▶ solved together in the Deep Dive

from the original lab

Challenge B — new

`suspect_card.dd`

`files/suspect_card.dd`

- ▶ a shipping office's USB stick
- ▶ FAT16, raw dd, 1.44 MB
- ▶ one entry starts with 0xE5...
- ▶ recover what deletion forgot

deleted-file recovery

Challenge C — new

`mini_mft.bin`

`files/mini_mft.bin`

- ▶ 16 crafted MFT records
- ▶ one file's times disagree
- ▶ follow parents, read residents
- ▶ the story the kernel kept

timestomping hunt

Flag format: `ctf{xxxx}` (Challenge A has no flag - its answers are the artifacts)

Working paper & evidence rules

1 Pre-lab first

section 1.4 of the working paper, before the session

2 Hash everything you export

every carved or exported file gets logged with its hash

3 Cite records and offsets

answers name the MFT record, cluster, or hex offset used

4 Screenshot the tool view

the tree + properties pane, not a paraphrase

5 Never analyze the original

copies only - and never image a device you own... casually

```
PS C:\Tools>  
  
# evidence log entry  
[B-05] 2026-10-26 10:03 FTK Imager  
      suspect_card.dd / root / 0xE5 entry  
      first char E5, FAT chain zeroed  
      -> evidence/B-05_deleted_entry.png
```

Bytes, not stories.

every claim cites a record, cluster, or offset

KEY TAKEAWAYS

01

Image, then analyze

bit-by-bit copies with hash verification keep evidence admissible

02

The filesystem keeps a diary

\$MFT records hold names, parents, times - even for deleted files

03

Deleted is not gone

0xE5 entries and freed MFT records still point at real bytes

04

Two clocks catch liars

SI times are faked easily; FN times are the kernel's memory

Copy the disk. Interrogate the copy.



PS C:\Tools>

```
PS> git clone https://github.com/stprasst/digital-forensic-lab
PS> cd digital-forensic-lab\Lab\ 06\files
PS> dir
Image.ad1  suspect_card.dd  mini_mft.bin
```